HW 4.3 Document

Nakul Bhatia

2026-04-08

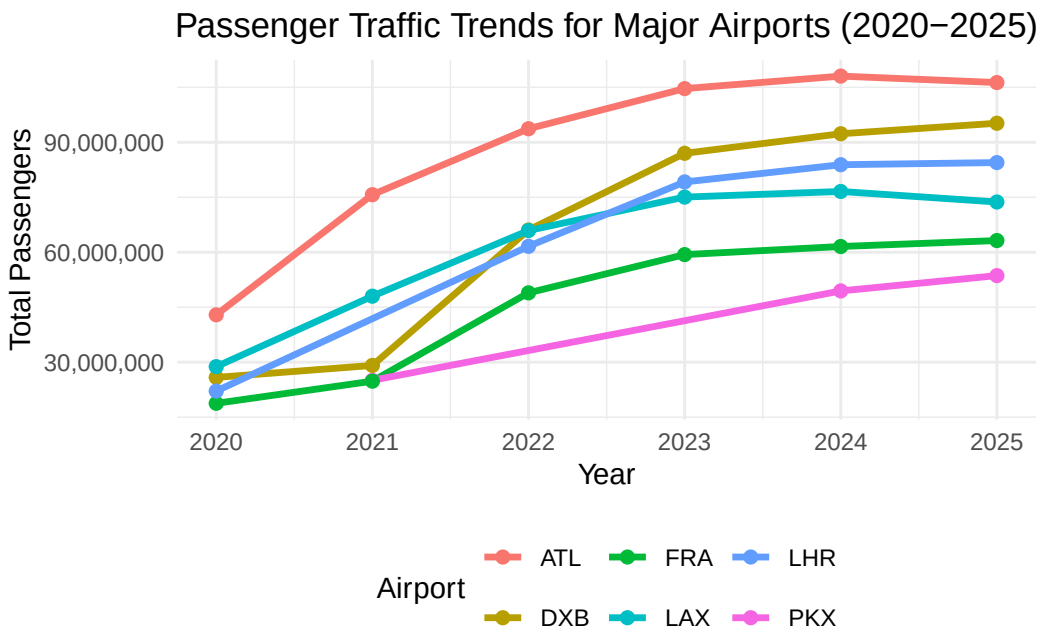
Busiest Airport Analysis

The table below summarizes passenger totals for the six selected airports from 2020 through 2025.

Table 1: Passenger totals for the six selected airports

airport	2021	2024	2025	2020	2022	2023
PKX	25051012	49441029	53618949	NA	NA	NA
DXB	29110609	92331506	95200000	25836771	66069981	86994365
FRA	24812849	61564957	63189666	18768601	48918482	59355389
ATL	75704760	108067766	106302208	42918685	93699630	104653451
LHR	NA	83884572	84463000	22111469	61614508	79183364
LAX	48007284	76588028	73709594	28779527	65924298	75050875

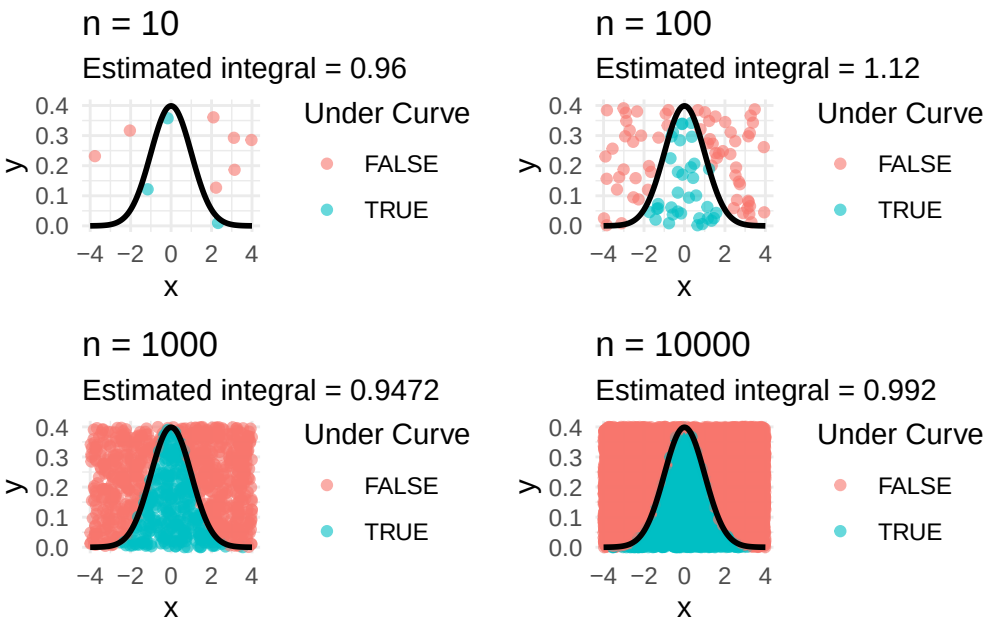
The table below demonstrates the number of passengers that traveled through each of these major airports from 2020 until 2025. Each of these airports experienced declines in the number of their passengers during the year 2020, but experienced an increase in the number of their passengers between 2021 and 2025. The busiest airport during this time period was Atlanta. Airports like Dubai also experienced growth in their passengers between 2020 and 2024, while Heathrow saw an increase in their passengers during 2025. Thus, each of these airports indicates the disruption that occurred in the year 2020 in relation to the pandemic, as well as the growth in the number of passengers that traveled to these airports following the pandemic outbreak.



The table and graph shows clear trends in passenger traffic across the six major airports from 2020 to 2025. All airport started off at a low point in 2020, most likely due to covid. This was then followed by a steady growth in the following years as covid went away. Hartsfield-Jackson Atlanta (ATL) consistently had the highest passenger numbers throughout the time period. Dubai (DXB) and London Heathrow (LHR) also grew strongly, with noticeable increases after 2021. Beijing Daxing (PKX) and Frankfurt (FRA) have lower overall passenger totals, but still show gradual increases over time. The graph makes it easy to compare trends across airports, while the table provides the exact values of passengers each year making it easier to directly compare. Overall, the data shows that global air travel has been recovering steadily since Covid, with different major international airports growing at different rates.

Monte Carlo Numerical Integration

The figure below shows Monte Carlo estimates of the area under the standard normal density curve on the interval from -4 to 4 using four different sample sizes.



The small multiple shows how the Monte Carlo estimate improves as the number of sampled points increases. With a small sample size (n = 10), the estimate varies widely and is not very accurate. As the number of points increases to 1000 and 10000, the points more closely fill the region under the curve and the estimate stabilizes. The exact value of the integral for the standard normal distribution over the interval [-4, 4] is very close to one, since almost all of the probability mass lies within this range. The plots with the larger sample sizes supports this, as the estimated values converge closer to 1 as the sample since increases. Also, the proportion of points under the curve becomes more consistent as sample size gets bigger. This shows that increasing the number of random samples improves the accuracy and reliability of the Monte Carlo integration.

Planning and Prompting Gen AI Tools

My plan is to: 1. Access the mtcars dataset through R's built-in datasets function to view its contents.

2. Use the str() or glimpse() function to observe the structure of the mtcars data frame to become familiar with the different variables that comprise the data set and the data type of each of those variables.
3. Determine which variables will be utilized to analyze the data within the mtcars data set - the variables of interest are the mpg and hp variables of the data set.

4. Use the select() function within the dplyr package to select only those variables of interest from the mtcars data set.
5. Use the is.na() and sum() functions to determine if there are any missing values within the data set that may impact its integrity and cleanliness.
6. Use the class() function within R to determine the class of each of the mpg and hp variables to ensure they are of the numeric data type, as is required by the scatter plot function within the ggplot2 package.
7. Create a data set that has been cleaned of any variables that were not of interest to the analysis yet which were included within the mtcars data set.
8. Utilize the graph types that will best represent the data - a scatter plot will be utilized to visualize the relationship between the two variables of interest within the data set.
9. Create a scatter plot utilizing the ggplot2 package that graphs the hp variable against the mpg variable within the data set.
10. Increase the readability of the scatter plot by adding a title, labels for each of the axes, and a plotting theme.
11. Interpret the scatter plot that was created with the mtcars data set to gain an understanding of the differences between the mpg and hp variables for each of the cars included within the data set.
12. Use the prepared and cleansed data set and the scatter plot that was created with the data to formulate the prompt that will be utilized to request that the GenAI tool also create this same graph.

AI Generated Plan Prompt: "# Load the built-in mtcars data set data(mtcars)

View the data

mtcars

Examine the structure of the data

str(mtcars)

Load dplyr

library(dplyr)

Select only the variables of interest

mtcars_clean <- mtcars %>% select(mpg, hp)

Check for missing values

sum(is.na(mtcars_clean))

Check the class of each variable

class(mtcars_clean$mpg)$class(mtcars_clean$hp)

Load ggplot2

library(ggplot2)

Create scatter plot

ggplot(mtcars_clean, aes(x = hp, y = mpg)) + geom_point() + labs(title = "Scatter Plot of Horsepower vs. Miles Per Gallon", x = "Horsepower (hp)", y = "Miles Per Gallon (mpg)") + theme_minimal()"

AI Generated Simple Prompt: "Nice, this is a classic dataset to work with — let's walk through exactly how to do it in R.

Step 1: Load the dataset

mtcars is already built into R, so you don't need to download anything.

data(mtcars)

If you want to see it:

View(mtcars) Step 2: Load ggplot2

If you haven't installed it yet:

install.packages("ggplot2")

Then load it:

library(ggplot2) Step 3: Create a basic plot

Let's make a scatter plot of mpg (fuel efficiency) vs hp (horsepower):

ggplot(data = mtcars, aes(x = hp, y = mpg)) + geom_point() Step 4: Make it look better

Add labels and a title:

ggplot(data = mtcars, aes(x = hp, y = mpg)) + geom_point(color = "blue") + labs(title = "MPG vs Horsepower", x = "Horsepower", y = "Miles per Gallon") Step 5 (Optional): Add a trend line

This helps show the relationship:

ggplot(data = mtcars, aes(x = hp, y = mpg)) + geom_point(color = "blue") + geom_smooth(method = "lm", se = FALSE, color = "red") + labs(title = "MPG vs Horsepower with Trend Line", x = "Horsepower", y = "Miles per Gallon") What this shows (you can use in your assignment) There is a negative relationship between horsepower and MPG As horsepower increases, fuel efficiency decreases"

The results generated from the plan-based prompt were more effective and better aligned with the intended analysis compared to the generic prompt. By clearly outlining the dataset, variables, and goals in the plan-based prompt, the AI was able to produce code that directly addressed the relationship between horsepower and miles per gallon. The output was structured, easy to follow, and required little to no modification.

In contrast, the generic prompt produced a more general response that lacked specificity. While it still provided a valid approach to creating a visualization, it did not focus as directly on the variables of interest and required additional adjustments to fully match the intended analysis. The lack of detail in the generic prompt resulted in a broader and less tailored solution.

This comparison highlights the importance of carefully planning prompts when using generative AI tools. A well-structured and detailed prompt leads to more accurate, relevant, and efficient results, while a vague prompt can produce outputs that require additional refinement. Overall, the plan-based prompt demonstrated that providing clear instructions significantly improves the quality and usability of AI-generated code.

Reflection

Throughout this course, I have developed a much stronger understanding of data wrangling, visualization, and working with R. At the beginning, I found it challenging to understand how to clean and reshape data, especially when working with functions like pivot_longer() and pivot_wider(). However, through repeated practice in assignments, I became more comfortable using these tools to transform data into a tidy format.

I also improved my ability to create clear and effective visualizations using ggplot2. Earlier in the course, I focused mainly on getting code to work, but now I am more aware of how to make graphs more readable and informative by adding labels, titles, and appropriate formatting.

Another important takeaway from this course was learning how to think about data analysis as a process. Instead of jumping straight into coding, I now understand the importance of planning, identifying relevant variables, and structuring the workflow before beginning the analysis. This was especially evident in the GenAI section, where creating a clear plan led to better results from the AI tool.

Overall, this course helped me build confidence in working with data and using R. While there were challenges along the way, especially with debugging and understanding errors, I now feel more capable of approaching data analysis problems in a structured and effective way.

Appendix A: Gen AI Usage

Using ChatGPT Version 5.3 4/15/2026 Prompt: 1. Access the mtcars dataset through R's built-in datasets function to view its contents. 2. Use the str() or glimpse() function to observe the structure of the mtcars data frame to become familiar with the different variables that comprise the data set and the data type of each of those variables. 3. Determine which variables will be utilized to analyze the data within the mtcars data set - the variables of interest are the mpg and hp variables of the data set. 4. Use the select() function within the dplyr package to select only those variables of interest from the mtcars data set. 5. Use the is.na() and sum() functions to determine if there are any missing values within the data set that may impact its integrity and cleanliness. 6. Use the class() function within R to determine the class of each of the mpg and hp variables to ensure they are of the numeric data type, as is required by the scatter plot function within the ggplot2 package. 7. Create a data set that has been cleaned of any variables that were not of interest to the analysis yet which were included within the mtcars data set. 8. Utilize the graph types that will best represent the data - a scatter plot will be utilized to visualize the relationship between the two variables of interest within the data set. 9. Create a scatter plot utilizing the ggplot2 package that graphs the hp variable against the mpg variable within the data set. 10. Increase the readability of the scatter plot by adding a title, labels for each of the axes, and a plotting theme. 11. Interpret the scatter plot that was created with the mtcars data set to gain an understanding of the differences between the mpg and hp variables for each of the cars included within the data set. 12. Use the prepared and cleansed data set and the scatter plot that was created with the data to formulate the prompt that will be utilized to request that the GenAI tool also create this same graph. Using the mtcars data in R do the plan. Generated: "Absolutely — here's a clean plan using the built-in mtcars data in R, written so you can use it in your assignment. Data Analysis Plan Using mtcars in R I will first access the built-in mtcars data set in R by using the data() function and then display it on the screen to view its contents. This will allow me to confirm that the data set has loaded properly and to become familiar with the observations being analyzed. Next, I will use the str() function, or alternatively glimpse() from the dplyr package, to examine the structure of the mtcars data frame. This step will help me identify the variables included in the data set and determine the data type of each variable. After reviewing the structure of the data, I will determine which variables

are most relevant to the analysis. For this project, the variables of interest are mpg (miles per gallon) and hp (horsepower), since I want to examine the relationship between fuel efficiency and engine power. I will then load the dplyr package and use the select() function to keep only the mpg and hp variables from the original mtcars data set. This will make the data easier to work with by removing variables that are not needed for the analysis. Once the relevant variables have been selected, I will check the data for missing values by using is.na() together with sum(). This step is important because missing values could affect the accuracy of the graph and the interpretation of the results. I will also use the class() function to confirm that both mpg and hp are numeric variables. This is necessary because a scatter plot in ggplot2 requires numeric data for both the x- and y-axes. After confirming that the selected variables are usable and free of missing values, I will create a cleaned data set containing only mpg and hp. This cleaned version of the data will be used for the graph and for any interpretation of the relationship between the two variables. To best represent the relationship between two numeric variables, I will use a scatter plot. A scatter plot is appropriate because it shows how one quantitative variable changes in relation to another quantitative variable. Using the ggplot2 package, I will create a scatter plot with hp plotted on the x-axis and mpg plotted on the y-axis. This will visually display the relationship between horsepower and miles per gallon for the cars in the mtcars data set. To improve the readability and presentation of the graph, I will add a descriptive title, clear axis labels, and a theme such as theme_minimal(). These additions will make the graph easier to understand and more visually polished. After creating the scatter plot, I will interpret the graph by looking for patterns, trends, or unusual points. In particular, I will examine whether cars with higher horsepower tend to have lower miles per gallon, which would suggest a negative relationship between the two variables. Finally, I will use the cleaned data set and the completed graph as the basis for a prompt to a GenAI tool. The prompt will ask the tool to recreate the same scatter plot in R using the mtcars data, the ggplot2 package, and the same formatting choices. R Code for the Plan # Load the built-in mtcars data set data(mtcars)

View the data

mtcars

Examine the structure of the data

str(mtcars)

Load dplyr

library(dplyr)

Select only the variables of interest

mtcars_clean <- mtcars %>% select(mpg, hp)

Check for missing values

sum(is.na(mtcars_clean))

Check the class of each variable

class(mtcars_clean$mpg)$class(mtcars_clean$hp)

Load ggplot2

library(ggplot2)

Create scatter plot

ggplot(mtcars_clean, aes(x = hp, y = mpg)) + geom_point() + labs(title = "Scatter Plot of Horsepower vs. Miles Per Gallon", x = "Horsepower (hp)", y = "Miles Per Gallon (mpg)") + theme_minimal() Interpretation

The scatter plot will likely show a negative relationship between horsepower and miles per gallon. This means that cars with higher horsepower generally tend to get lower gas mileage, while cars with lower horsepower tend to get better fuel efficiency. This pattern makes sense because more powerful engines often require more fuel."

Using ChatGPT Version 5.3 4/15/2026 Prompt: Use the mtcars data in R to make a plot using that data Generated: "Nice, this is a classic dataset to work with — let's walk through exactly how to do it in R.

Step 1: Load the dataset

mtcars is already built into R, so you don't need to download anything.

data(mtcars)

If you want to see it:

View(mtcars) Step 2: Load ggplot2

If you haven't installed it yet:

install.packages("ggplot2")

Then load it:

library(ggplot2) Step 3: Create a basic plot

Let's make a scatter plot of mpg (fuel efficiency) vs hp (horsepower):

ggplot(data = mtcars, aes(x = hp, y = mpg)) + geom_point() Step 4: Make it look better

Add labels and a title:

ggplot(data = mtcars, aes(x = hp, y = mpg)) + geom_point(color = "blue") + labs(title = "MPG vs Horsepower", x = "Horsepower", y = "Miles per Gallon") Step 5 (Optional): Add a trend line

This helps show the relationship:

ggplot(data = mtcars, aes(x = hp, y = mpg)) + geom_point(color = "blue") + geom_smooth(method = "lm", se = FALSE, color = "red") + labs(title = "MPG vs Horsepower with Trend Line", x = "Horsepower", y = "Miles per Gallon") What this shows (you can use in your assignment) There is a negative relationship between horsepower and MPG As horsepower increases, fuel efficiency decreases"

{{< pagebreak >}} ## Appendix B: Code Appendix

```
library(rvest)
library(dplyr)
library(purrr)
library(janitor)
library(tidyr)
library(ggplot2)

url <- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic#2024_sta
page <- rvest::read_html(url)

nodes <- rvest::html_elements(page, css = "table.wikitable")
wiki_tables <- purrr::map(nodes, \(x) rvest::html_table(x))

years <- 2025:2020

clean_airport_table <- function(tbl, yr) {
  tbl <- clean_names(tbl)

  passenger_col <- names(tbl)[grepl("passenger", names(tbl), ignore.case = TRUE)]
```

```
  airport_col <- names(tbl)[grepl("airport", names(tbl), ignore.case = TRUE)]

  tbl |>
    mutate(year = yr) |>
    rename(
      total_passengers = all_of(passenger_col[1]),
      airport = all_of(airport_col[1])
    ) |>
    mutate(
      total_passengers = as.character(total_passengers),
      total_passengers = gsub("\\[.*?\\]", "", total_passengers),
      total_passengers = gsub(",", "", total_passengers),
      total_passengers = trimws(total_passengers),
      total_passengers = as.numeric(total_passengers)
    )
}

airport_data_2020_2025 <- map2_dfr(wiki_tables[1:6], years, clean_airport_table)

selected_airports <- airport_data_2020_2025 |>
  filter(grepl("Atlanta|Frankfurt|Daxing|Los Angeles|Dubai|Heathrow", airport)) |>
  select(year, airport, total_passengers) |>
  arrange(airport, year)

selected_airports_short <- selected_airports |>
  mutate(
    airport = recode(
      airport,
      "Hartsfield-Jackson Atlanta International Airport" = "ATL",
      "Frankfurt Airport" = "FRA",
      "Beijing Daxing International Airport" = "PKX",
      "Los Angeles International Airport" = "LAX",
      "Dubai International Airport" = "DXB",
      "Heathrow Airport" = "LHR"
    )
  )

airport_table <- selected_airports_short |>
  pivot_wider(
    names_from = year,
    values_from = total_passengers
  )
```

```r
airport_plot <- ggplot(selected_airports_short, aes(x = year, y = total_passengers, color = a
  geom_line(linewidth = 1.2) +
  geom_point(size = 2) +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "Passenger Traffic Trends for Major Airports (2020-2025)",
    x = "Year",
    y = "Total Passengers",
    color = "Airport"
  ) +
  theme_minimal() +
  theme(
    legend.position = "bottom",
    plot.title = element_text(hjust = 0.5)
  )
knitr::kable(airport_table, caption = "Passenger totals for the six selected airports")
airport_plot
library(dplyr)
library(ggplot2)
library(patchwork)

monte_carlo_points <- function(n, x_min, x_max, y_min, y_max) {
  x <- runif(n, min = x_min, max = x_max)
  y <- runif(n, min = y_min, max = y_max)

  data.frame(
    x = x,
    y = y
  )
}

make_mc_plot <- function(n) {
  points_df <- monte_carlo_points(
    n = n,
    x_min = -4,
    x_max = 4,
    y_min = 0,
    y_max = 0.4
  )

  mc_results <- points_df %>%
    mutate(
```

```
      f_x = dnorm(x, mean = 0, sd = 1),
      under_curve = y <= f_x
    )

  rectangle_area <- (4 - (-4)) * (0.4 - 0)

  estimated_integral <- mc_results %>%
    summarize(
      proportion_under = mean(under_curve),
      estimated_area = proportion_under * rectangle_area
    )

  ggplot(mc_results, aes(x = x, y = y, color = under_curve)) +
    geom_point(alpha = 0.6) +
    stat_function(
      fun = dnorm,
      args = list(mean = 0, sd = 1),
      xlim = c(-4, 4),
      linewidth = 1,
      inherit.aes = FALSE
    ) +
    labs(
      title = paste("n =", n),
      subtitle = paste("Estimated integral =", round(estimated_integral$estimated_area, 4)),
      x = "x",
      y = "y",
      color = "Under Curve"
    ) +
    coord_cartesian(xlim = c(-4, 4), ylim = c(0, 0.4)) +
    theme_minimal()
}

plot10 <- make_mc_plot(10)
plot100 <- make_mc_plot(100)
plot1000 <- make_mc_plot(1000)
plot10000 <- make_mc_plot(10000)

mc_small_multiple <- plot10 + plot100 + plot1000 + plot10000
mc_small_multiple
library(rvest)
library(dplyr)
library(purrr)
```

```r
library(janitor)
library(tidyr)
library(ggplot2)

url <- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic#2024_sta
page <- rvest::read_html(url)

nodes <- rvest::html_elements(page, css = "table.wikitable")
wiki_tables <- purrr::map(nodes, \(x) rvest::html_table(x))

years <- 2025:2020

clean_airport_table <- function(tbl, yr) {
  tbl <- clean_names(tbl)

  passenger_col <- names(tbl)[grepl("passenger", names(tbl), ignore.case = TRUE)]
  airport_col <- names(tbl)[grepl("airport", names(tbl), ignore.case = TRUE)]

  tbl |>
    mutate(year = yr) |>
    rename(
      total_passengers = all_of(passenger_col[1]),
      airport = all_of(airport_col[1])
    ) |>
    mutate(
      total_passengers = as.character(total_passengers),
      total_passengers = gsub("\\[.*?\\]", "", total_passengers),
      total_passengers = gsub(",", "", total_passengers),
      total_passengers = trimws(total_passengers),
      total_passengers = as.numeric(total_passengers)
    )
}

airport_data_2020_2025 <- map2_dfr(wiki_tables[1:6], years, clean_airport_table)

selected_airports <- airport_data_2020_2025 |>
  filter(grepl("Atlanta|Frankfurt|Daxing|Los Angeles|Dubai|Heathrow", airport)) |>
  select(year, airport, total_passengers) |>
  arrange(airport, year)

selected_airports_short <- selected_airports |>
  mutate(
```

```
    airport = recode(
      airport,
      "Hartsfield-Jackson Atlanta International Airport" = "ATL",
      "Frankfurt Airport" = "FRA",
      "Beijing Daxing International Airport" = "PKX",
      "Los Angeles International Airport" = "LAX",
      "Dubai International Airport" = "DXB",
      "Heathrow Airport" = "LHR"
    )
  )

airport_table <- selected_airports_short |>
  pivot_wider(
    names_from = year,
    values_from = total_passengers
  )

airport_plot <- ggplot(selected_airports_short, aes(x = year, y = total_passengers, color = a
  geom_line(linewidth = 1.2) +
  geom_point(size = 2) +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "Passenger Traffic Trends for Major Airports (2020-2025)",
    x = "Year",
    y = "Total Passengers",
    color = "Airport"
  ) +
  theme_minimal() +
  theme(
    legend.position = "bottom",
    plot.title = element_text(hjust = 0.5)
  )
library(dplyr)
library(ggplot2)
library(patchwork)

monte_carlo_points <- function(n, x_min, x_max, y_min, y_max) {
  x <- runif(n, min = x_min, max = x_max)
  y <- runif(n, min = y_min, max = y_max)

  data.frame(
    x = x,
```

```
    y = y
  )
}

make_mc_plot <- function(n) {
  points_df <- monte_carlo_points(
    n = n,
    x_min = -4,
    x_max = 4,
    y_min = 0,
    y_max = 0.4
  )

  mc_results <- points_df %>%
    mutate(
      f_x = dnorm(x, mean = 0, sd = 1),
      under_curve = y <= f_x
    )

  rectangle_area <- (4 - (-4)) * (0.4 - 0)

  estimated_integral <- mc_results %>%
    summarize(
      proportion_under = mean(under_curve),
      estimated_area = proportion_under * rectangle_area
    )

  ggplot(mc_results, aes(x = x, y = y, color = under_curve)) +
    geom_point(alpha = 0.6) +
    stat_function(
      fun = dnorm,
      args = list(mean = 0, sd = 1),
      xlim = c(-4, 4),
      linewidth = 1,
      inherit.aes = FALSE
    ) +
    labs(
      title = paste("n =", n),
      subtitle = paste("Estimated integral =", round(estimated_integral$estimated_area, 4)),
      x = "x",
      y = "y",
      color = "Under Curve"
```

```
    ) +
    coord_cartesian(xlim = c(-4, 4), ylim = c(0, 0.4)) +
    theme_minimal()
}

plot10 <- make_mc_plot(10)
plot100 <- make_mc_plot(100)
plot1000 <- make_mc_plot(1000)
plot10000 <- make_mc_plot(10000)

mc_small_multiple <- plot10 + plot100 + plot1000 + plot10000
```

```
library(dplyr)
library(ggplot2)
library(patchwork)

monte_carlo_points <- function(n, x_min, x_max, y_min, y_max) {
  x <- runif(n, min = x_min, max = x_max)
  y <- runif(n, min = y_min, max = y_max)

  data.frame(
    x = x,
    y = y
  )
}

make_mc_plot <- function(n) {
  points_df <- monte_carlo_points(
    n = n,
    x_min = -4,
    x_max = 4,
    y_min = 0,
    y_max = 0.4
  )

  mc_results <- points_df %>%
    mutate(
      f_x = dnorm(x, mean = 0, sd = 1),
      under_curve = y <= f_x
    )

  rectangle_area <- (4 - (-4)) * (0.4 - 0)
```

```r
  estimated_integral <- mc_results %>%
    summarize(
      proportion_under = mean(under_curve),
      estimated_area = proportion_under * rectangle_area
    )

  ggplot(mc_results, aes(x = x, y = y, color = under_curve)) +
    geom_point(alpha = 0.6) +
    stat_function(
      fun = dnorm,
      args = list(mean = 0, sd = 1),
      xlim = c(-4, 4),
      linewidth = 1,
      inherit.aes = FALSE
    ) +
    labs(
      title = paste("n =", n),
      subtitle = paste("Estimated integral =", round(estimated_integral$estimated_area, 4)),
      x = "x",
      y = "y",
      color = "Under Curve"
    ) +
    coord_cartesian(xlim = c(-4, 4), ylim = c(0, 0.4)) +
    theme_minimal()
}

plot10 <- make_mc_plot(10)
plot100 <- make_mc_plot(100)
plot1000 <- make_mc_plot(1000)
plot10000 <- make_mc_plot(10000)

mc_small_multiple <- plot10 + plot100 + plot1000 + plot10000
```